Contents

Introduction 3

Defining the Problems 4

Brain Tumour Segmentation 4

Domain Shifts 5

Choosing a Base Model 7

Convolutional Neural Networks (CNN) 7

Fully Convolutional Neural Networks (FCN) 8

U-Net 10

nnU-Net 11

Building a Base Model 13

Source Code 13

Model Training 14

Final Model 17

Test-Time Training (TTT) 18

Understanding TTT 18

Implementing TTT 19

Experiments with new architecture 19

TTT Results 21

Evaluation 22

A Note of Gratitude 22

Challenges 22

Assessment of Objectives 23

References 24

Appendix 26

Appendix A. Project Code 26

Appendix B. Base Model Training Experiment Logs 27

Appendix C. A little bit of fun 47

Introduction

To treat brain tumours, radiologists observe MRI scans to determine their location in a process called segmentation.[1] Manual methods can be time-consuming and prone to human error. There is therefore motivation to automate this process using machine learning.[2]

When I was building software for brain tumour segmentation for my Computer Science A-level coursework, I noticed a few key issues. First, my clustering algorithms were quite inaccurate. Whilst researching, I repeatedly came across papers using neural networks to achieve better performance. This was something I really wanted to try.

Furthermore, when I was interviewing a doctor for my project, he asked how my model would adapt to hospitals with different scanner types. I did not have any measures in place to tackle this, and I was unsettled by how dangerous that was.

Given I had limited time with my CS coursework, the senior scholarship was a great opportunity for me to address my dissatisfaction and take on these two problems. I contacted a professor at the Computer Intelligence Lab at Rowan University who agreed to mentor me and provide computing resources for this project.

Defining the Problems

Brain Tumour Segmentation

Segmentation involves assigning each individual part of an input a specific label. These label assignments can be stored in an array called a segmentation mask. For image segmentation, an example of which is shown in Figure 1, each "individual part" is a pixel.

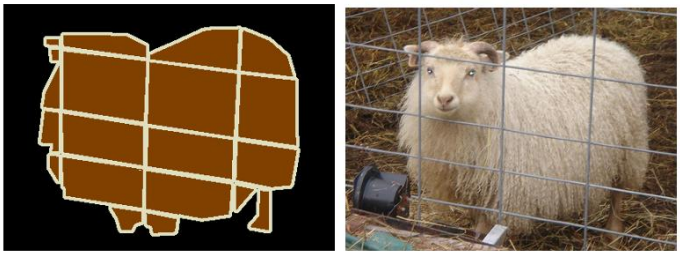


Figure 1. Example of image segmentation. Each pixel is classified as non-sheep or sheep. On the right side is the original image and on the left side is the segmentation mask. [3]

Meanwhile, brain tumours are segmented from 3D MRI scans, which are made up of voxels[i] instead of pixels. Usually, four different MRI modalities[ii] are needed: T1, T1ce (T1 contrast), T2 and FLAIR, since brain tumours consist of different subregions that have different intensities on each scan modality.[2]

My goal is to train a neural network that takes 3D MRI scans as input and produces a segmentation mask that identifies the different tumour regions. Examples of such masks are shown in Figure 2.

[i] Voxel: The building block of 3D volumes. They can be understood as 3D pixels
[ii] Modality: A specific type of MRI scan

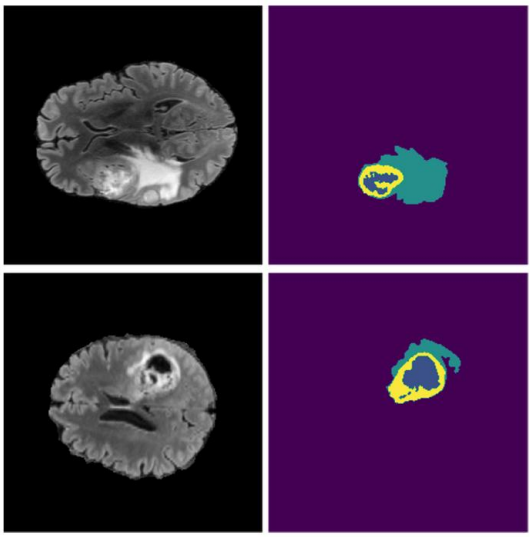


Figure 2. An example of brain tumour segmentation. On the left a single slice of the FLAIR modality MRI scan is shown. On the right are the brain tumour segmentation masks with each colour representing a different tumour subregion. [4]

Domain Shifts

Data in the same domain share similar feature distributions. The main purpose of any machine learning algorithm is essentially to learn this shared distribution. Hence, a model's performance is very likely to deteriorate when used on a different distribution. In the context of brain tumours, this might involve a change in hospital scanners or patient demographic. If not regulated, wrong predictions could become more likely.

One way this problem can be tackled is with a technique called Test-Time Training (TTT), which I decided to implement. My first requirement is to train a base model before adding features necessary for TTT.

The BraTS (**Bra**in **T**umour **S**egmentation) challenge is a yearly competition that provides a golden-standard data for brain tumour segmentation. I had previously only encountered the datasets from 2021 and earlier, which only provided Glioma[iii] samples. To see what potential domain shifts were present, I explored datasets available in later years.

[iii] Glioma: A specific type of brain tumour

First, I considered using samples from different hospitals, each of which have different scanning equipment. However, I realised that there was no one hospital that had enough data to train the base model.

Then, I considered using pre-operative vs post-operative scans. However, the set of tumour subregions for each domain are different as post-operative scans have a resection cavity. This meant the labels for each domain were different, which is strictly not allowed for TTT.[5]

I settled on using a tumour type shift. This was ideal since BraTS provides the BraTS-MEN and BraTS-Africa datasets. BraTS-MEN contains 1,000 samples of the Meningioma tumour type, which I could use to train the base model. The BraTS-Africa dataset then provides data for both the Glioma and "Other Neoplasms" tumour types, which I could use as the domain shifted dataset. Importantly, these two datasets have the same set of labels, i.e. the NETC[iv], SNFH[v] and ET[vi] tumour subregions.[7]

[iv] NETC: Stands for non-enhancing tumour core. Represents dead or non-growing tissue within the tumour core[6]

[v] SNFH: Stands for surrounding non-enhancing FLAIR hyperintensity. Represents areas where tumour cells have invaded into healthy brain tissue[6]

[vi] ET: Stands for enhancing tumour. Represents tumour tissue[6]

Choosing a Base Model

Next, I needed to choose an architecture for my base model. I began by considering the CNN, as it provides a foundation to many deep learning approaches to image analysis. However, I identified several limitations that made the standard CNN incompatible with my objectives. I discovered that the FCN model, which builds on CNN architecture, solves these problems. I then researched the U-Net, a form of FCN specialised for medical imaging tasks. Finally, I settled on the nnU-Net, which builds upon the U-Net architecture to a self-adapting framework to automate network configuration for any given dataset.

Convolutional Neural Networks (CNN)

The CNN starts with an input layer and is followed by a series of convolutional and pooling layers known as the encoder. These use kernels[vii] to extract important features from input data. The data is then passed to a set of fully connected layers.

On the first fully connected layer, the data is flattened into a 1D array, meaning that this layer's size depends directly on the size of the input data. This is problematic for my project as it requires all inputs of the CNN to be the same size. However, I am using two datasets each with different input sizes.

The standard CNN is also unsuitable for image segmentation, where each pixel must be assigned a label. As the input was passed through the encoder, the size of its feature maps[viii] was reduced by pooling layers. By the time it is flattened into a 1D array, its spatial structure has been lost and cannot be reconstructed using the fully connected layers. A standard CNN is therefore more suitable for classification tasks such as the

[vii] Kernel: Matrices of weights that act like a filter to slide over an image and extract important features
[viii] Feature map: The output of a convolutional or pooling layer

one shown in Figure 3.[8]

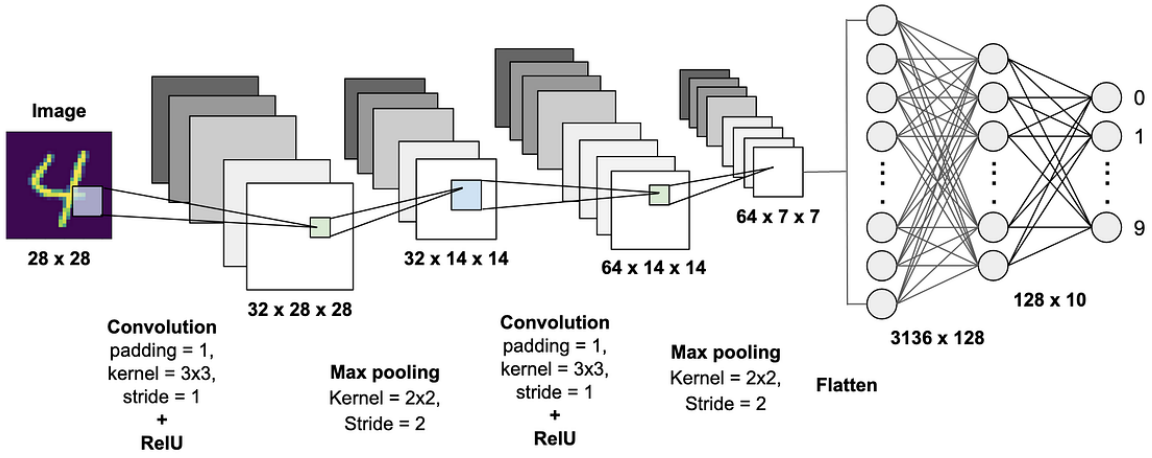


Figure 3. An example of the architecture of a CNN used to identify the number in the input image. The output is a set of 10 probabilities, each representing the likelihood of the input image being a number from 0 - 9 [9]

Fully Convolutional Neural Networks (FCN)

The FCN architecture resolves both problems by not using fully connected layers and adding a decoder. To receive inputs of any size, the flattening function is not used. Instead, FCN uses a bottleneck that can take inputs or arbitrary size, as shown in Figure 4.

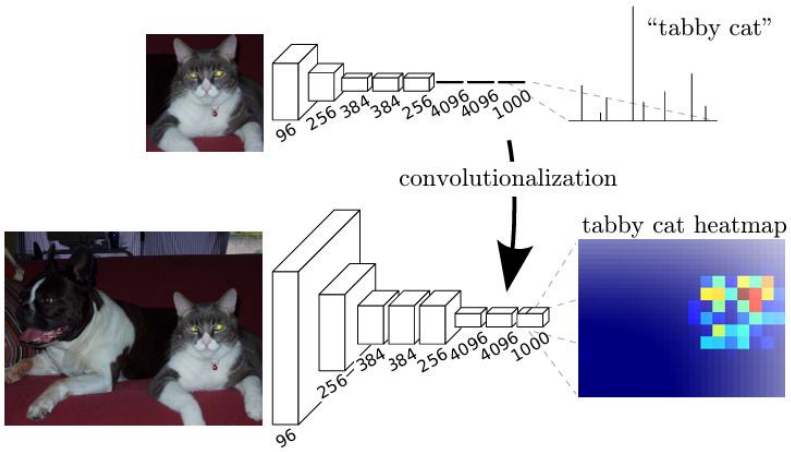


Figure 4. Shows the difference between the CNN encoder (top) and FCN encoder (bottom). In the second image, the input data is no longer flattened. Also, the output of the bottleneck is a heatmap instead of a set of probabilities. [3]

Additionally, the use of a decoder right after the bottleneck means the FCN can perform pixel-wise segmentation. The decoder reconstructs the output from the bottleneck to

the resolution of the original input image to form a segmentation mask. It does this using a series of convolutional and up-sampling layers. These up-sampling layers reverse the effect of pooling layers in the encoder by increasing the size of feature maps. This is often done through transpose convolutions, which are explained in Figure 5.

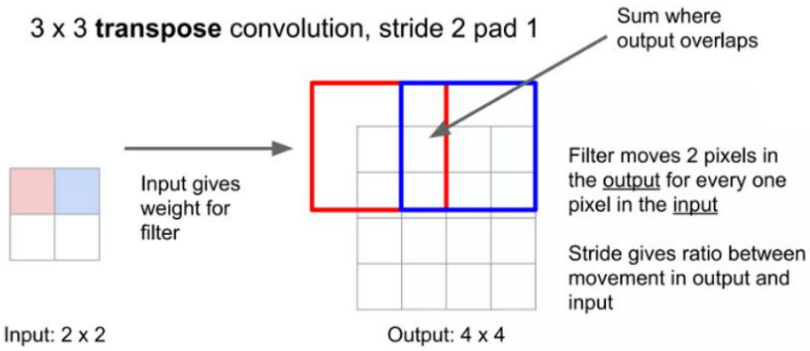


Figure 5. An example of transpose convolutions. [10]

However, when up-sampling is performed, some spatial information[ix] is lost as we are trying to move from a lower to higher resolution. The decoder mainly holds semantic information[x] about the class labels whilst the encoder holds spatial information about what the original input image looked like. Hence, it would be beneficial to combine information from both sections during up-sampling using a technique called skip connections. On each level of the decoder, a prediction is produced and combined with the corresponding encoder feature map, as shown in Figure 6. The benefits of using it are shown in Figure 7. Figure 8 shows the full FCN architecture.

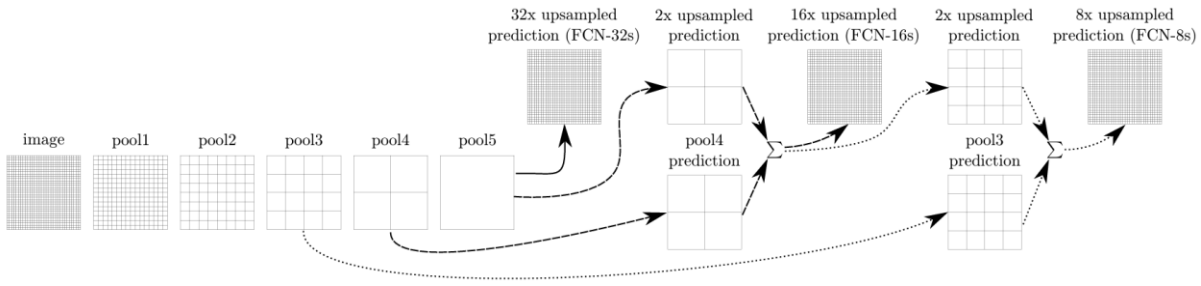


Figure 6. An example of how skip connections are used. On each level of the decoder, a prediction is produced and combined with the corresponding feature map on the encoder. [3]

[ix] Spatial information: Information about the shape, location and structure of objects in an image
[x] Semantic information: Information about the identity of objects in an image

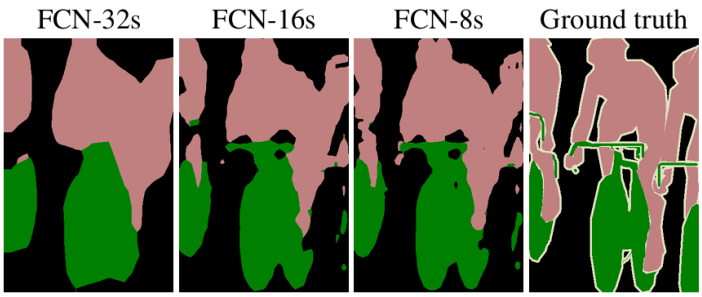


Figure 7. An example of the benefits of skip connections. FCN-32 represents the prediction on the first level of the decoder. FCN-16 represents the prediction on the next layer, and FCN-8 represents the next. [3]

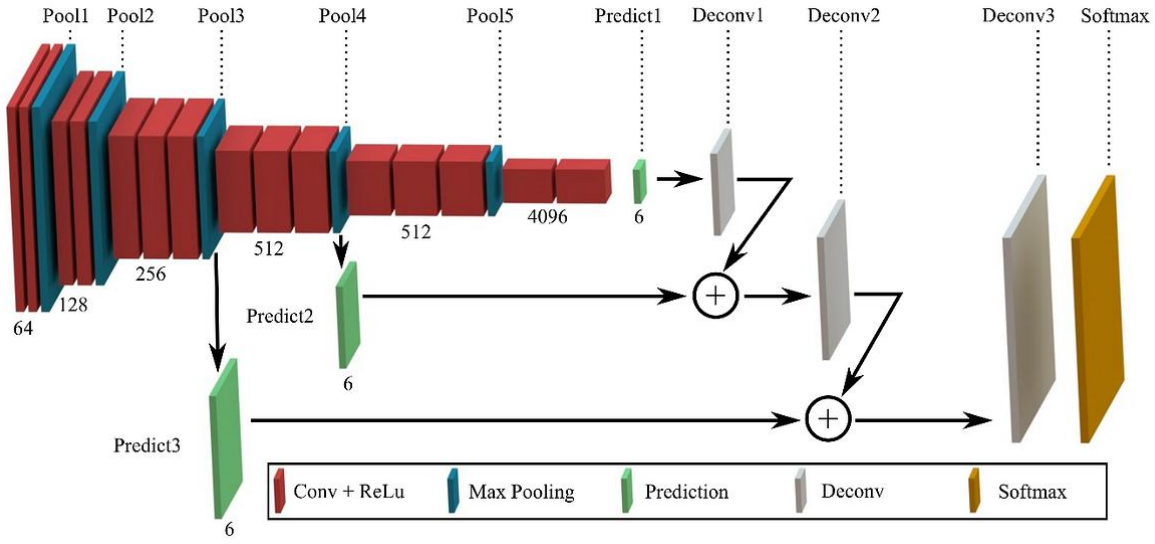


Figure 8. An example of the full FCN architecture. [11]

U-Net

The FCN architecture is very strong for general segmentation. However, my task is specifically medical imaging segmentation, which requires a much higher level of precision given that many biological structures are very small. Hence, it is very important that when reconstructing the segmentation mask using the decoder, fine-grained spatial information must be retained to ensure that no important details are lost.

The U-Net builds upon the FCN architecture and is specialised for medical imaging segmentation. By implementing an entirely symmetrical encoder and decoder, skip connections can be used on every level of the model. This allows the reconstruction

mask to be much more spatially accurate compared to the original input image. Figure 9 shows an example of the U-net architecture.

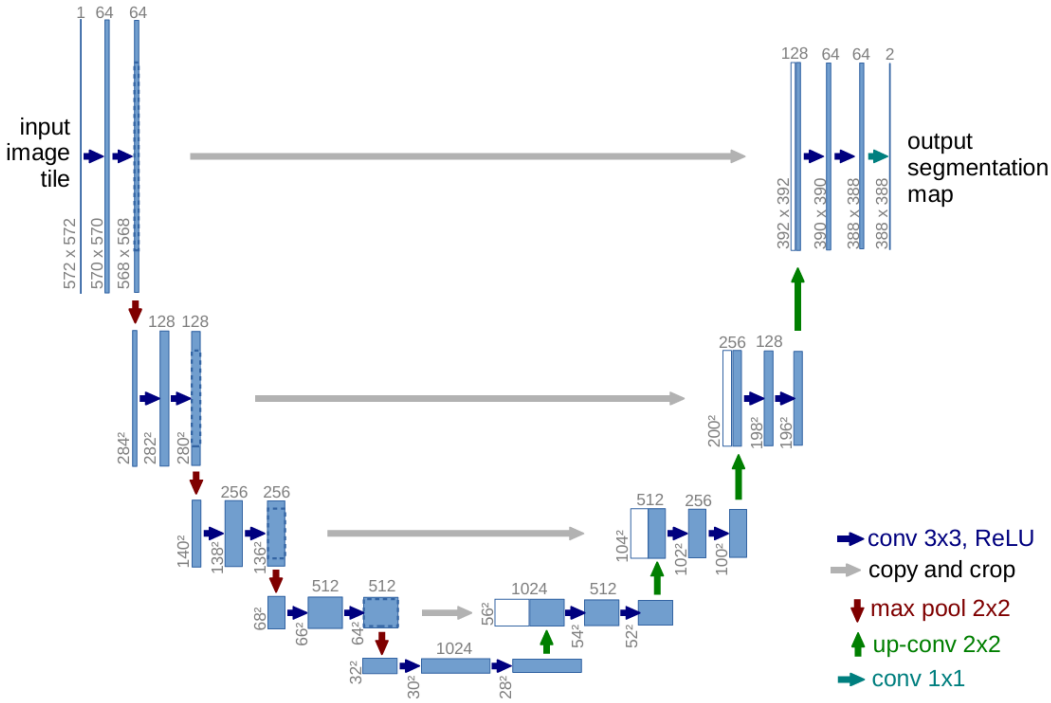


Figure 9. An example of the full U-Net architecture. [12]

nnU-Net

As I continued my research, I came across a large number of different U-Net versions such as the UNETR[13] and SegResNetVAE[14] models. They were each optimised in a different way and I felt somewhat lost regarding which one to choose. However, I came across the nnU-Net (which stands for no new U-Net) paper by Ronneberger et al. which argues that that often, overly specialised architectures and training schemes achieve competitive performance on a specific task but fail to generalise well. Hence, they argue that the basic U-Net is the best model to use.[15] The Optimised U-Net for Brain Tumour Segmentation paper by Futrega et al. also used ablation studies[xi] to corroborate the U-Net being better than its variants for my project objective.[4]

In the end, I decided to use nnU-Net architecture for my base model. It adds to the basic U-Net model by building around it a framework that automatically adapts the

[xi] Ablation studies: The process of removing a component from a system and studying the performance to determine its contribution[16]

network and training pipeline depending on features of the given dataset such as voxel spacings[xii] and image dimensions. Some features that it automatically configures include the input patch[xiii] size, kernel sizes, stride sizes and preprocessing pipeline.

[xii] Voxel spacings: The distance between centres of adjacent voxels[17]
[xiii] Patch: To reduce the size of input images to a network, it can be split into separate parts, each of which is called a patch and processed separately

Building a Base Model

Source Code

At first, I attempted to build the nnU-Net pipeline from scratch and even managed to build a U-Net model using Python. However, I realised it was unrealistic to implement the entire pipeline in the time given. Luckily, whilst re-reading the Optimised U-Net paper by Futrega et al. I noticed that they had open sourced their code on GitHub and their repository contained a full nnU-Net pipeline[4]. Meanwhile, I have always wanted to work with professional open-source code. Hence, I decided to use their code as a base, adding things and making changes where necessary.

The challenge that came with this decision was understanding the code. There was virtually no documentation, and the programme structure was quite complicated. I had also approached understanding the nnU-Net from a different perspective compared to the author, which made the logic of their code difficult to comprehend.

For instance, I would have constructed the U-Net sequentially, starting with the encoder, then bottleneck and decoder as shown on the left in Figure 10. However, the author used a recursive[xiv] implementation to construct it layer by layer as shown on the right.[18]

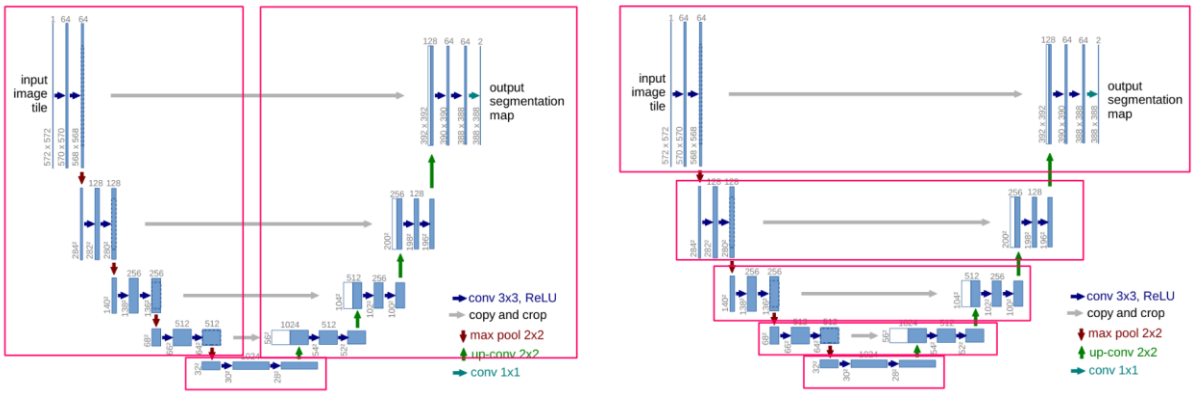


Figure 10. My understanding of how the U-Net should be built (left) vs the author of the NVIDIA's nnU-Net open source code's understanding (right)

[xiv] Recursive: A method that is defined in terms of itself

Furthermore, there were some functionalities the code did not include which I added separately, such as plotting the loss and dice graphs as well as getting and plotting model predictions. To do this, I needed a complete and strong understanding of the existing programmes.

Model Training

I trained the model through a series of experiments (which I have called "runs"), each time making a change to the model architecture or training configuration and checking if model performance had increased. I judged model performance using an overall dice score as well as dice scores per tumour subregion.

To determine what sort of changes could improve model performance, one of the things I analysed was the training and validation loss. Validation refers to a small subset of the training data that is not revealed to the model and can be used as testing data during training to check model performance on an unseen dataset.

Figure 11 shows the loss plot for Run 01, for which I used a training scheme of 30 epochs[xv]. I noticed that the training and validation loss were both still changing, indicating that there was still room for improvement. I increased the number of epochs to 150 and in Run 03, the losses had converged better.

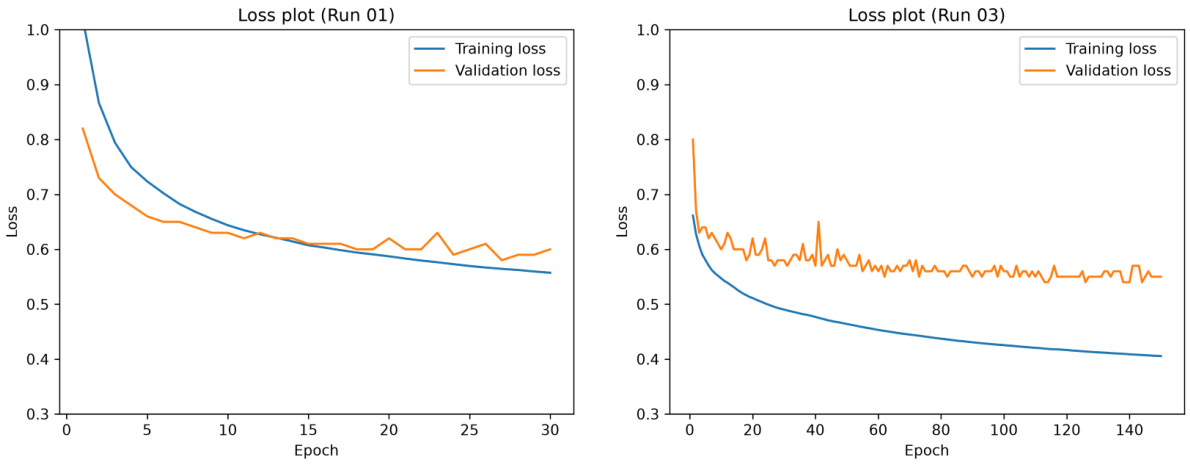


Figure 11. Loss plots for two different model training runs

[xv] Epochs: The number of complete times a complete dataset makes a pass through a network during training

I analysed the model's predictions by plotting them alongside the ground truth label[xvi] and original input. Most of the time, as shown in Figure 12, the model performs well. However, sometimes it fails to detect very small tumours, as shown in Figure 13. It also tends to fail on local regions as shown in Figure 14. Meanwhile, the model is not affected by outliers. An example of what I mean by outliers is shown in Figure 15.

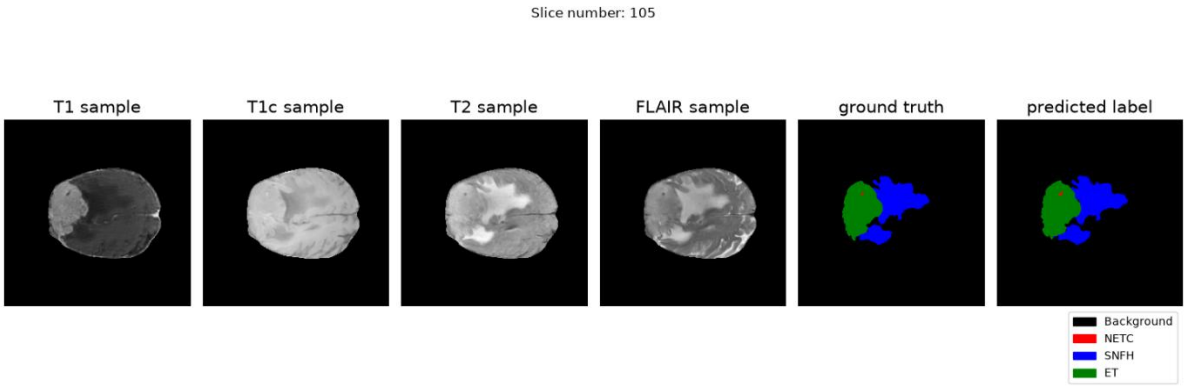


Figure 12. An example of the model's good performance. From the left to the right are the four input scans, the ground truth label, and the model prediction.

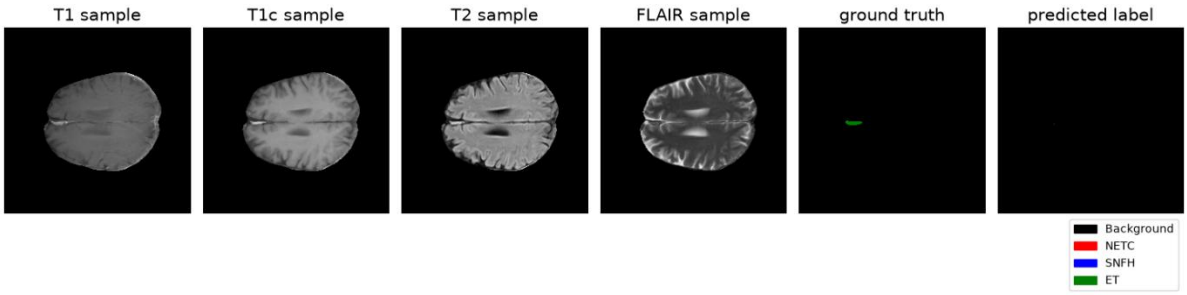


Figure 13. An example of the model failing to detect very small tumours. From the left to the right are the four input scans, the ground truth label, and the model prediction.

[xvi] Ground truth: The true value

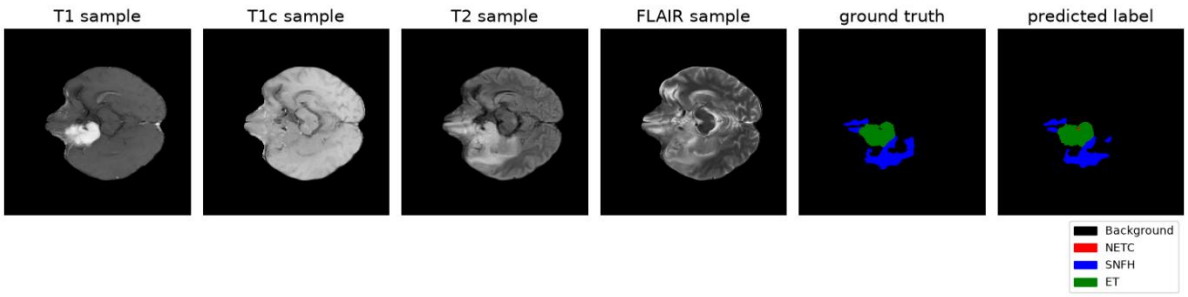


Figure 14. An example of the model failing on local regions. From the left to the right are the four input scans, the ground truth label, and the model prediction.



Figure 15. An example of my K-Means clustering algorithm from my CS coursework being affected by outliers. My prediction is on the right whilst the ground truth is shown on the left.

Hence, given that the model struggled with local features, I increased the filter sizes of the U-Net's top layers.

The nnU-Net code also came with two model variants: BraTS22 and DynUNet (Dynamic U-Net). After running tests on both, I found that the DynUNet consistently performed better. I was also able to add ResBlocks (Residual Blocks) to this model, which helps prevent gradient vanishing[xvii] [20] and/or gradient explosion[xviii] [20] for large models like mine.

I kept detailed logs of my experiments, which can be found in Appendix B.

[xvii] Gradient vanishing: A common problem for large neural networks that causes gradients to become extremely small during training[19]
[xviii] Gradient explosion: A common problem for large neural networks that causes gradients to become excessively large during training[19]

Final Model

The performance of my final model on the training set is shown in Table 1. I was satisfied with this result since it was much better than the model I had for my CS coursework, which achieved a dice score barely over 0.50.

	Dice	Val loss	Train loss	D1	D2	D3	Best epoch
Final model	75.30	0.247	0.3758	70.52	77.56	77.83	132/150

Table 1. Shows the performance metrics of my final model. D1, D2 and D3 are the dice scores of the NETC, SNFH and ET tumour subregions.

I had trained the model on a subset of BraTS-MEN data, and my next task was for the model to generalise to BraTS-Africa Glioma and BraTS-Africa Other Neoplasms on which I had hypothesised model performance would decrease. I now checked this hypothesis using my new base model. And indeed, Table 2 confirms a significant drop in performance across both new domains.

	D1	D2	D3	Average
BraTS-MEN	75.7	75.10	73.80	74.90
BraTS-Africa Other Neoplasms	29.52	35.56	76.70	47.30
BraTS-Africa Glioma	37.30	45.15	67.50	50.00

Table 2. The performance metrics of my final model on each different dataset.

Test-Time Training (TTT)

Understanding TTT

My base model currently only performs well on Meningioma data. if a patient with a different tumour type arrives, the model needs to adapt to this new domain. However, we have a big problem: the sample data in the new domain comes with no label. Without knowing the right answer, how can we train the model?

The solution is self-supervision. We can apply a function to the input and tell our model to guess what the original image was. Through this process, the model learns the new domain's feature distribution.

Hence, I introduced a new branch that performs a reconstruction[xix] task. It sits parallel to the main task branch, which produces the segmentation mask, and uses the same decoder structure (without skip connections). The reconstruction and main task share the encoder and bottleneck. I adapted this technique from the Test-Time Training with Self-Supervision for Generalization under Distribution Shifts paper by Sun et al.[5] My understanding of the model architecture is shown in Figure 16.

[xix] Reconstruction: Recreating the original input

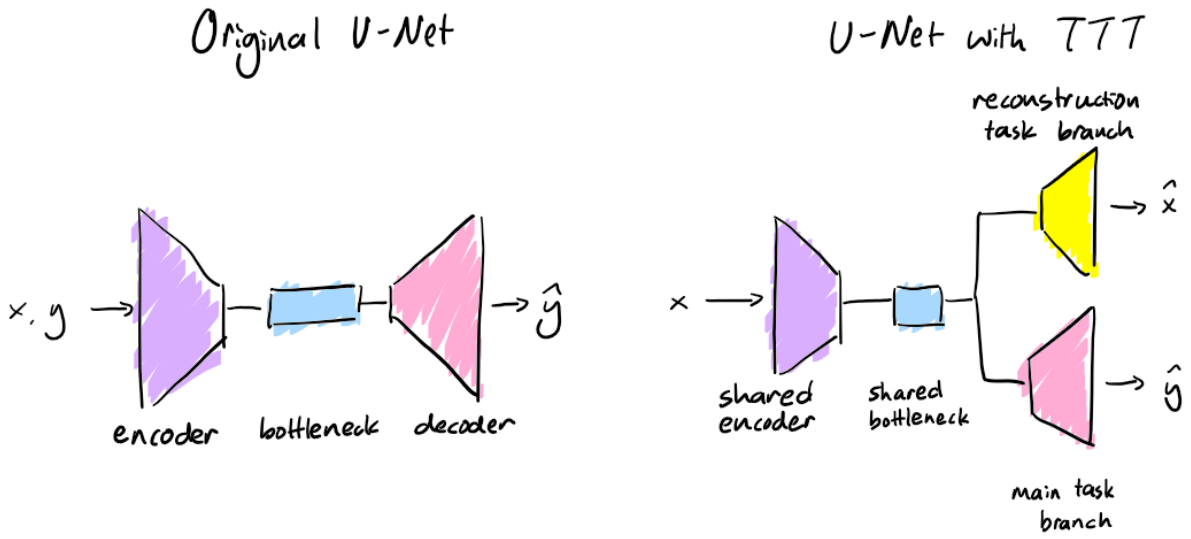


Figure 16. My diagrams of the original U-Net architecture (left) and the new U-Net with TTT architecture (right). x is the input MRI scan, and y is the ground truth label. x̂ represents the reconstruction and ŷ represents the segmentation mask.

Implementing TTT

To implement TTT, I re-programmed the DynUNet class, adding a reconstruction task branch parallel to the main task branch. I edited the nnU-Net pipeline so that reconstruction loss was used during training.

Then, I added the testing with TTT pipeline, described in Figure 17, to the original nnU-Net pipeline.[5]

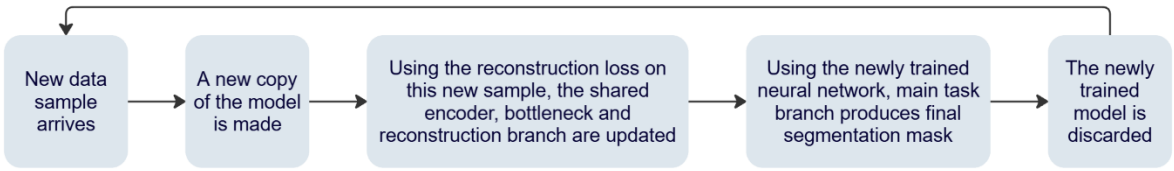


Figure 17. Flow chart showing the TTT testing pipeline

Experiments with new architecture

I first ran a test to make sure the newly added reconstruction branch had not caused model performance to drop. I did this by training a new model and testing it on the BraTS-MEN data without using the TTT testing pipeline. This produced a similar dice score compared to original model version, as shown in the last row of Table 3.

As shown in Table 3, I had trained this model with a reconstruction loss weight of 1.0. This weight indicates the importance of the reconstruction loss compared to the main task loss during training. Next, I tuned this weight by slowly increasing its magnitude from 1e-9. At the end, I chose to use a weight of 1.0 as it produced the least blurry reconstructions without sacrificing performance. Examples of the reconstructions produced by the model are shown in Figure 18.

	Weight of reconstruction task loss	**Dice score**	**Notes**
Run 12	1e-9	Overall: 72.55; D1: 66.95; D2: 70.01; D3: 80.69	Was completely unable to predict the reconstruction
Run 13	1e-5	Overall: 72.59; D1: 70.06; D2: 66.77; D3: 80.94	Was only able to predict the foreground mask for the reconstruction
Run 14	0.001	Overall: 73.47; D1: 66.99; D2: 71.41; D3: 82.00	Was also only able to predict foreground mask for reconstruction
Run 15	0.1	Overall: 73.32; D1: 65.58; D2: 74.47; D3: 79.90	Was able to predict reconstruction, but was quite blurry
Run 16	0.5	Overall: 73.01; D1: 71.03; D2: 64.95; D3: 83.05	Was able to predict reconstruction, and is less blurry than Run 15
Run 17	1	Overall: 73.10; D1: 72.18; D2: 67.30; D3: 79.82	Was able to predict reconstruction, and is less blurry than Run 16

Table 3. Shows the model performance depending on weight of reconstruction task loss. See footnotes for definition of foreground mask[xx].

[xx] Foreground mask: a bounding box that tells the model where the brain is, allowing it to ignore the irrelevant background data

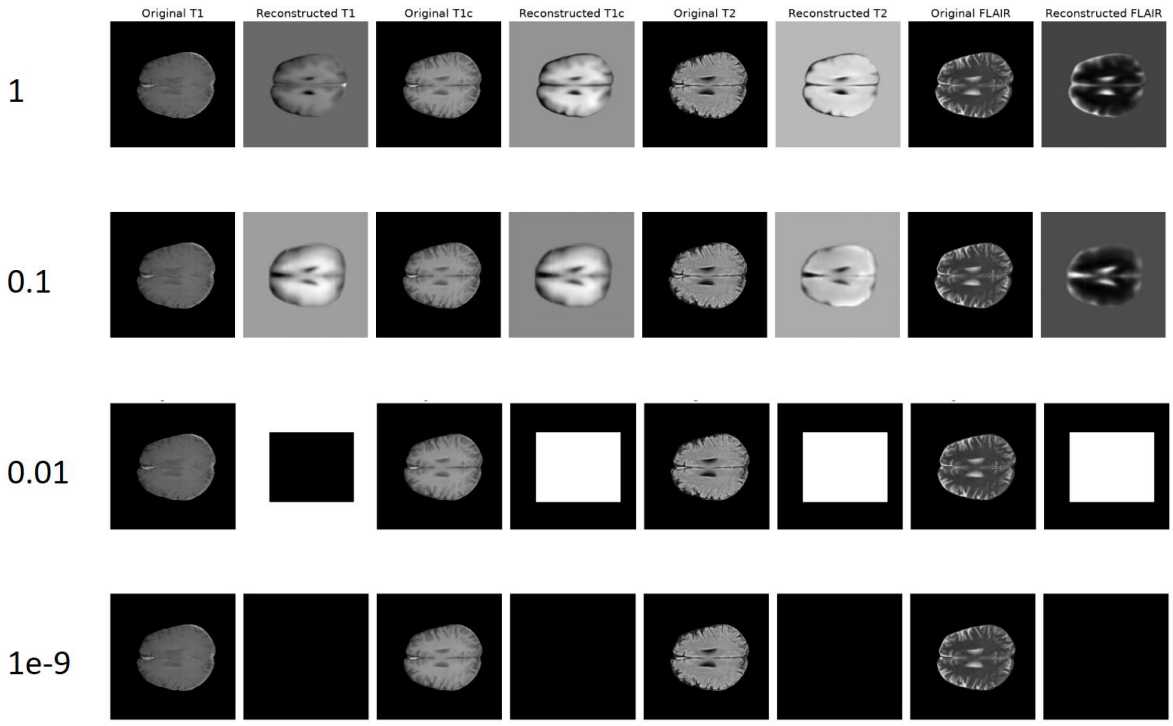


Figure 18. Shows an example of the reconstructions for each reconstruction task loss weight. Each consecutive pair of images represents the original scan (left) and the attempted reconstruction (right).

TTT Results

Finally, I implemented TTT during testing. The results, shown in Table 4, are clearly much better than without TTT, which was shown in Table 2.

	D1	D2	D3	Average
BraTS-MEN	77.40	65.30	77.04	73.24
BraTS-Africa Other Neoplasms	34.79	52.56	75.24	54.20
BraTS-Africa Glioma	49.65	61.65	72.17	61.16

Table 4. Performance metrics on each of the three different datasets with model using TTT

Evaluation

A Note of Gratitude

I am immensely grateful to everyone at the Computer Intelligence Lab (CIL) at Rowan University for their mentorship throughout the project and for giving me access to their computing resources, without which this project would not have been possible. I thank Dr Xiajun Jiang for his help not only with this project, but with helping me develop much stronger and much more rigorous research skills.

Challenges

One of the biggest challenges I faced at first was reading research papers. They often assumed substantial background knowledge, especially since many papers were from earlier years where often authors assumed readers were already experts. For instance, I spent a week working through the first autoencoder paper I read. However, the more I read, the more I could grasp the focus of papers and no longer be hindered by the dense jargon. Towards the end of the project, I was able to understand the TTT paper in just a few hours.

Another hurdle I came across was obtaining the BraTS data. Initially, I had planned to use data from 2024, the BraTS-MET dataset that contained Metastases tumour type samples. However, I soon discovered that most of the 2024 data, including BraTS-MET, had been removed due to a “government shutdown”. I was almost about to sign up for the 2026 BraTS challenge just to access the data when I found the BraTS-Africa datasets and pivoted the objective of my domain shift.

At first, I also struggled a lot with experimental research. I under-estimated the time it would take to train a model, thinking it would only take around 2 hours when in fact it took around 10. This meant each experiment was very precious, something I did not realise at first. I failed to log my experiments in detail and sometimes even forgot the purpose of a particular run. I also often jumped the gun, making too many changes for one experiment. This meant I could not tell what effect each individual change had on model performance, rendering the run practically useless. Much time was wasted this

way, but by the end of the project, I had learnt my lesson and my experiments were much more organised.

I also struggled with using open-source code, especially one that was so complex. As mentioned, I found it hard to understand especially since it lacked almost any form of documentation. Nevertheless, it was an extremely valuable experience as it showed me what professional code should look like. Working with it also helped me develop much stronger debugging skills as I was working with unfamiliar code. I often spent hours tracing through functions to locate the exact source of a problem, which felt a lot like a scavenger hunt.

Assessment of Objectives

My initial objectives were to 1) improve brain tumour segmentation accuracy using neural networks and 2) ensure my model could adapt to new domains, both of which I feel I have achieved relatively well. However, even though model accuracy on the new domains was higher with TTT than without, it was still lower compared to the original domain. Given that, I believe it would still be dangerous to deploy. There are still many techniques out there I haven't tried, such as meta-learning, and perhaps in the future I can experiment with these too.

Beyond that, my biggest aim for this project was to deepen my understanding of how AI can be used in the medical field and become a stronger researcher. And I can confidently say I have achieved both. This project was a huge milestone for me. I trained my first ever large-scale neural network this summer and used my first ever GPU to do that (and it was the new NVIDIA GeForce RTX 5090 at that!). It was my first time working with such complex open-source code and my first time doing research in a real lab! This was definitely an unforgettable experience and an invaluable learning journey.

References

(1) *Tests to diagnose brain tumours*. https://www.cancerresearchuk.org/about-cancer/brain-tumours/getting-diagnosed/what-are-the-tests (accessed 2026-08-29).

(2) Myronenko, A. 3D MRI Brain Tumor Segmentation Using Autoencoder Regularization. arXiv November 19, 2018. https://doi.org/10.48550/arXiv.1810.11654.

(3) Long, J.; Shelhamer, E.; Darrell, T. Fully Convolutional Networks for Semantic Segmentation. arXiv March 8, 2015. https://doi.org/10.48550/arXiv.1411.4038.

(4) Futrega, M.; Milesi, A.; Marcinkiewicz, M.; Ribalta, P. Optimized U-Net for Brain Tumor Segmentation. arXiv December 24, 2021. https://doi.org/10.48550/arXiv.2110.03352.

(5) Sun, Y.; Wang, X.; Liu, Z.; Miller, J.; Efros, A. A.; Hardt, M. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. arXiv July 1, 2020. https://doi.org/10.48550/arXiv.1909.13231.

(6) *BraTS Brain Tumor Segmentation*. https://www.emergentmind.com/topics/brain-tumor-segmentation-brats (accessed 2026-08-31).

(7) info@sagebase.org, S. B. *Brain TumorS (BraTS) Challenges*. https://www.synapse.org/Synapse:syn53708126/wiki/626320 (accessed 2026-08-29).

(8) Krizhevsky, A.; Sutskever, I.; Hinton, G. E. ImageNet Classification with Deep Convolutional Neural Networks. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc., 2012; Vol. 25.

(9) DIC, M. A. P. Ms. *Introduction to Convolutional Neural Networks*. Analytics Vidhya. https://medium.com/analytics-vidhya/introduction-to-convolutional-neural-networks-c50f41e3bc66 (accessed 2026-08-29).

(10) *Stanford University CS231n: Deep Learning for Computer Vision*. https://cs231n.stanford.edu/ (accessed 2026-08-29).

(11) Gaikwad, M. *Overview: Fully Convolutional Network for Semantic Segmentation*. Medium. https://medium.com/@mohit_gaikwad/overview-fully-convolutional-network-for-semantic-segmentation-b4ef92eeb8c4 (accessed 2026-08-29).

(12) Ronneberger, O.; Fischer, P.; Brox, T. U-Net: Convolutional Networks for Biomedical Image Segmentation. arXiv May 18, 2015. https://doi.org/10.48550/arXiv.1505.04597.

(13) Hatamizadeh, A.; Tang, Y.; Nath, V.; Yang, D.; Myronenko, A.; Landman, B.; Roth, H.; Xu, D. UNETR: Transformers for 3D Medical Image Segmentation. arXiv October 9, 2021. https://doi.org/10.48550/arXiv.2103.10504.

(14) Myronenko, A. 3D MRI Brain Tumor Segmentation Using Autoencoder Regularization. arXiv November 19, 2018. https://doi.org/10.48550/arXiv.1810.11654.

(15) Isensee, F.; Petersen, J.; Klein, A.; Zimmerer, D.; Jaeger, P. F.; Kohl, S.; Wasserthal, J.; Koehler, G.; Norajitra, T.; Wirkert, S.; Maier-Hein, K. H. nnU-Net: Self-Adapting Framework for U-Net-Based Medical Image Segmentation. arXiv September 27, 2018. https://doi.org/10.48550/arXiv.1809.10486.

(16) *6.2. Ablation Studies — Increase citations, ease review & collaboration: Better ML in Science*. https://ml.recipes/notebooks/6-ablation-study.html (accessed 2026-08-29).

(17) *Voxel Spacing: The Building Block of Digital Imaging*. Bohrium. https://www.bohrium.com/en/sciencepedia/feynman/keyword/voxel_spacing (accessed 2026-08-31).

(18) *DeepLearningExamples/PyTorch/Segmentation/nnUNet at master · NVIDIA/DeepLearningExamples*. GitHub. https://github.com/NVIDIA/DeepLearningExamples/tree/master/PyTorch/Segmentation/nnUNet (accessed 2026-08-29).

(19) *Vanishing and Exploding Gradients Problems in Deep Learning*. GeeksforGeeks. https://www.geeksforgeeks.org/deep-learning/vanishing-and-exploding-gradients-problems-in-deep-learning/ (accessed 2026-08-29).

(20) He, K.; Zhang, X.; Ren, S.; Sun, J. Deep Residual Learning for Image Recognition. arXiv December 10, 2015. https://doi.org/10.48550/arXiv.1512.03385.

Appendix

Appendix A. Project Code

My code for this project is currently only available on a private GitHub repository. Hopefully, it will be made public soon, in which case you will be able to access it using the following link: Shr00mp/Brain-Tumour-Seg-TTA: Test time adaptation for brain tumour segmentation neural network. If this link does not work, feel free to email me at s.wang@spgs.org for access.

Note that the datasets, trained model checkpoints and model outputs (albeit a few images) are not available on the GitHub repository as they are too large.

Appendix B. Base Model Training Experiment Logs

A log of my experiments for training the base model is shown below. Apologies in advance for any strange formatting as this was copied over from Google Docs.

Run 02

Run objectives and summary

- Increased the epoch size from 30 to 100
- Changed the training batch size to 4 from 2. Validation batch size is still 4
- Added deep supervision

Model Configuration

Backbone	**DynUNet**
input shape	(5, 145, 162, 133)
output channels	4 (for each of the tumour subregions and the background)
filters	[64, 96, 128, 192, 256, 384]
kernels	dynamically generated based on voxel spacings
strides	dynamically generated based on voxel spacings

Training Configuration

epoch number	100
batch size	4
validation batch size	4

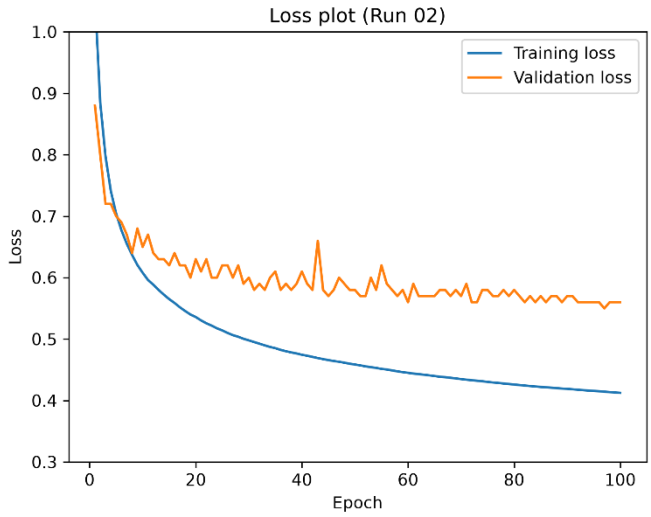
learning rate	0.0003
deep supervision	true
cross-fold validation	true, folds=5
optimiser	adam

Experiment results

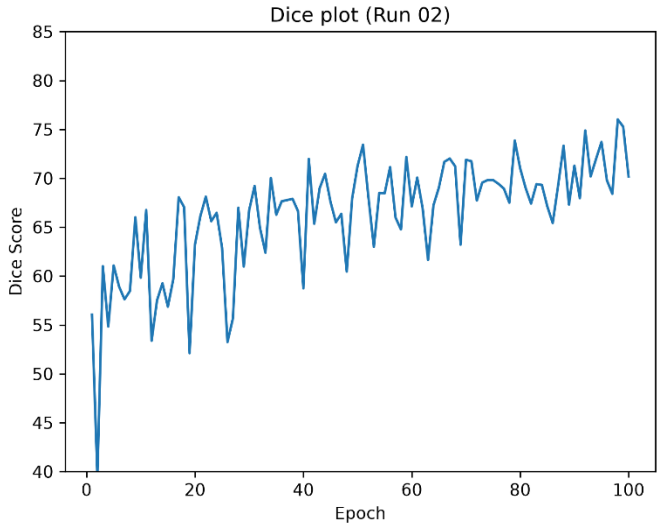
Performance metrics:

Dice	Val loss	Train loss	D1	D2	D3	Best epoch
76.02	0.56	0.4137	76.53	74.1	77.43	97/100

Loss plot:



Dice plot:



Run 03

Run objectives and summary

- The last run used the DynUNet model. For this run, we can try using the brats22 model
- Increased epoch size from 100 to 150
- It seems the BraTS model does not do as well compared to the DynUNet.

Model Configuration

Backbone	**BraTS 22 Model**
input shape	(5, 145, 162, 133)
output channels	3 (for each of the tumour subregions)
filters	Corrected: [64, 128, 256, 512, 768, 1024] (Incorrect: [64, 128, 256, 512, 768, 1024, 2048])
kernels	dynamically generated based on voxel spacings

strides	dynamically generated based on voxel spacings

Training Configuration

epoch number	150
batch size	2
validation batch size	4
learning rate	0.0003
deep supervision	true
cross-fold validation	true, folds=5
optimiser	adam

- I tried running this with training batch size 4, but it took up too much space in the memory and errored out immediately

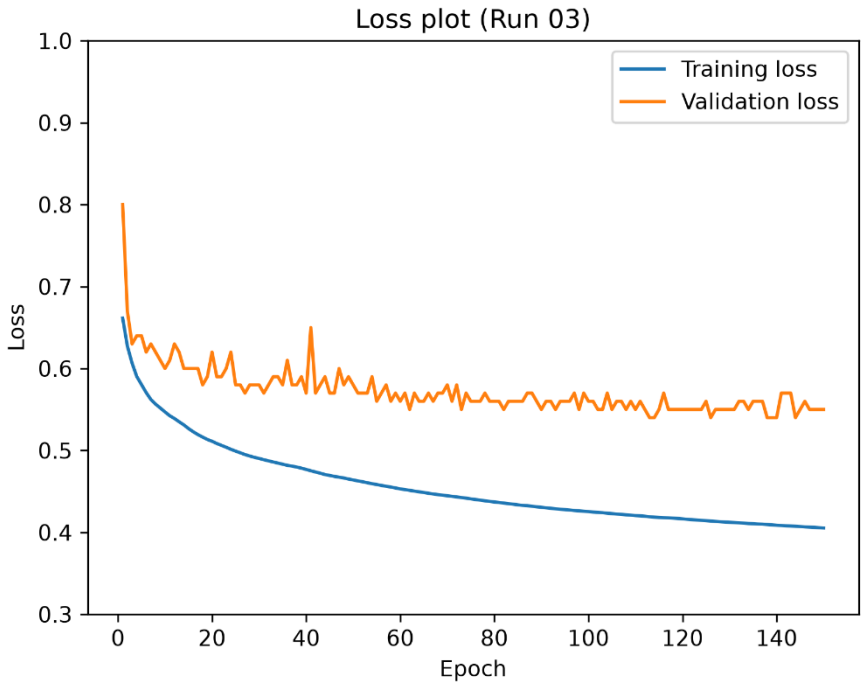
Experiment results

Dice	**Val loss**	**Train loss**	**D1**	**D2**	**D3**	**Best epoch**
72.13	0.56	0.4054	68.12	66.86	81.41	94/150

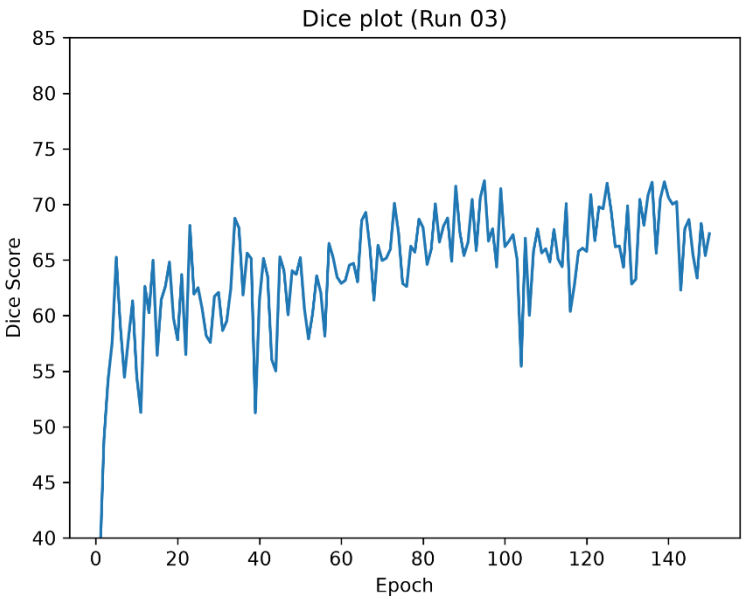
- The dice score for this model is less than the DynUNet model
- The validation loss is the same. However, the train loss is less compared to the last run

- Notably, the best epoch is epoch 94 out of 150 epochs. This means that for the last 56 epochs, there were no improvements in the dice score
- Compared to the DynUNet, brats model works well to identify D3 but does a lot worse on D1 and D2. The DynUNet seems to generalise a lot better over different subregion

Loss plot:



Dice plot:



Run 04

Run objectives and summary

- I ran the DynUNet model again, with the same training configuration compared to Run 02, except this time with 150 epochs to see if it could cause the model to improve
- The training and validation losses were significantly less compared to Run 02, but the dice score was very similar
- I also saved the last checkpoint for this model and used it to run predictions on 10 samples

Model Configuration

Backbone	**DynUNet**
input shape	(5, 145, 162, 133)
output channels	3 (for each of the tumour subregions)
filters	Corrected: [32, 64, 128, 256, 320, 320]
kernels	dynamically generated based on voxel spacings [[3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3]]
strides	dynamically generated based on voxel spacings [[1, 1, 1], [2, 2, 2], [2, 2, 2], [2, 2, 2], [2, 2, 2], [2, 2, 2]]

Training Configuration

epoch number	150

batch size	2
validation batch size	4
learning rate	0.0003
deep supervision	true
cross-fold validation	true, folds=5
optimiser	adam

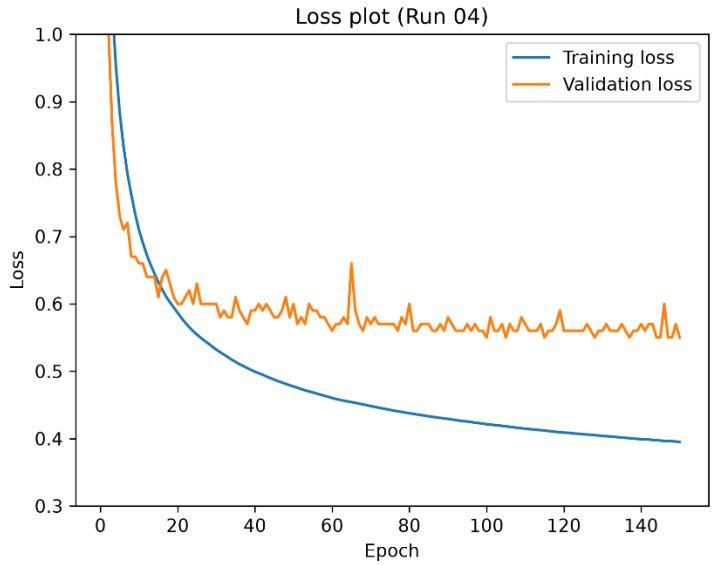
Experiment results

Metrics

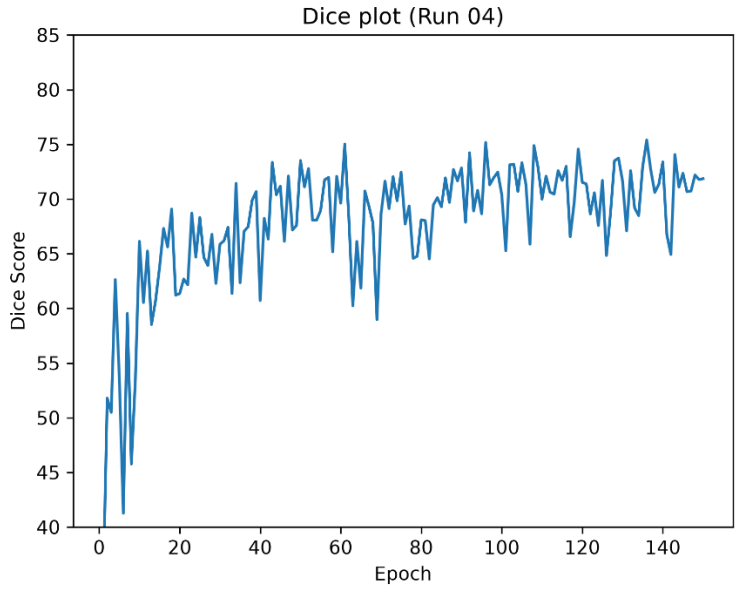
Dice	Val loss	Train loss	D1 (NETC)	D2 (SNFH)	D3 (ET)	Best epoch
75.40	0.246	0.3952	76.12	71.29	78.78	135/150

- The validation and train loss are both much less than in Run 02, but the dice score is basically the same

Loss plot:



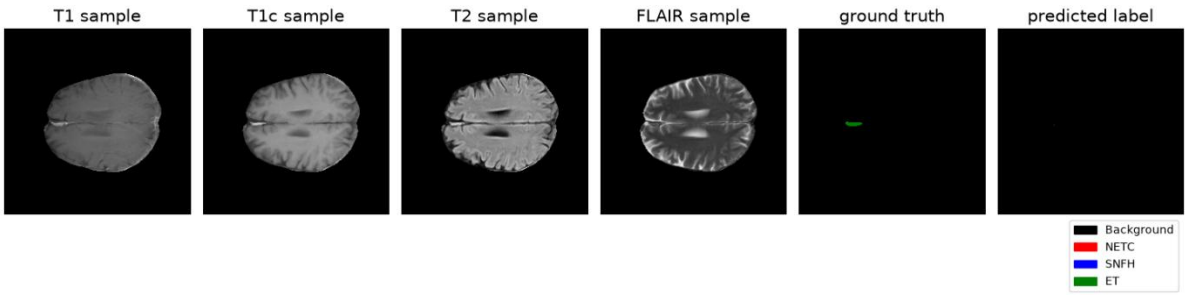
Dice plot:



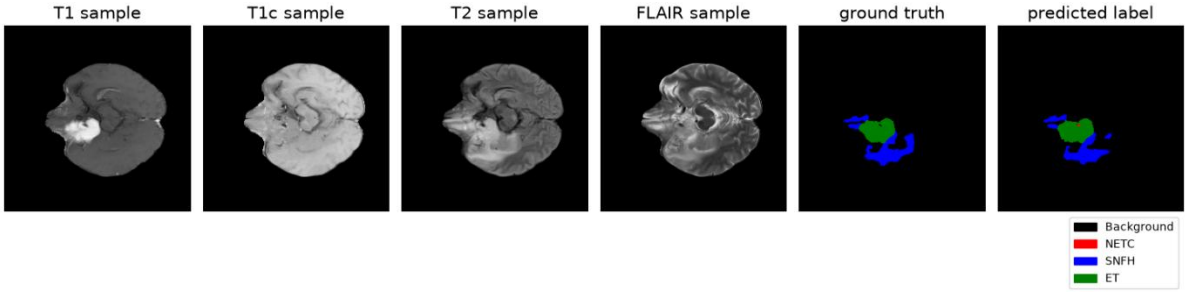
Predictions analysis

I took 10 random samples and plotted the original sample alongside the ground truth and model prediction. I noticed 2 main problems:

1. For some very small tumours, the model is unable to detect them, e.g.



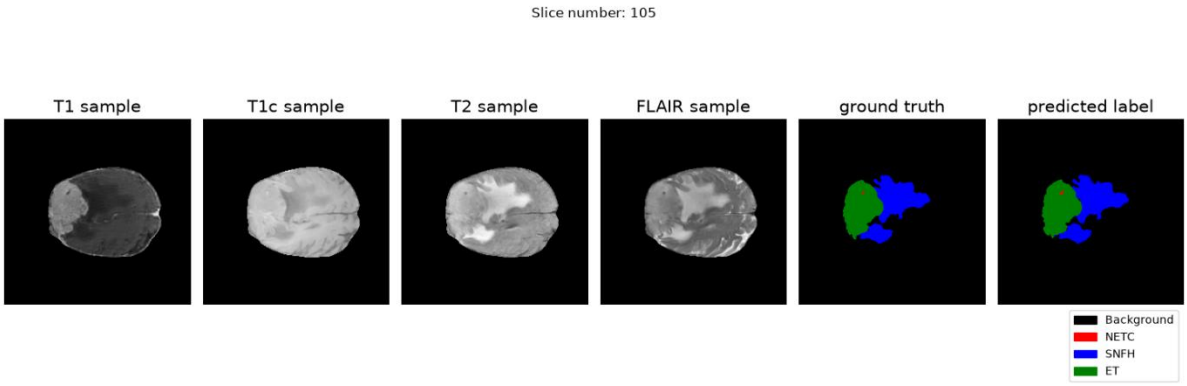
2. The model also sometimes struggles with missing local details, which I suppose is the same problem as 2). e.g.



These were the percentages of each subregion out of the total ROI (i.e. not including non-tumour regions). Since NETC is quite severely under-represented, it makes sense that D1 is usually quite low, especially compared to D3, which is for the most represented ET region.

NETC (red region)	2.4%
SNFH (blue region)	47.5%
ET (green region)	50.1%

I noticed that for samples where the tumour region is larger, the model seems to maybe do better compared to smaller tumour regions (although this could just be a problem with perspective)



Run 05

Run objectives and summary

- Add padding for the first layer so that the feature map size does not decrease
- I tested the BraTS 2022 model again, this time with reduced number of filters compared to the Run 03
    - Run 03 filters: [64, 128, 256, 512, 768, 1024]
    - Current run filters: [64, 96, 128, 192, 256, 384]
    - Comparing the performance, they are very similar, so it seems it is not a problem to reduce the number of filters
- After checking the predictions for this model, it seems it makes similar mistakes compared to the DynUNet model. It can get the general shape of the ROI but has trouble with some specifics.

Model Configuration

Backbone	**BraTS 22 Model**
input shape	(5, 145, 162, 133)
output channels	3 (for each of the tumour subregions)
filters	[64, 96, 128, 192, 256, 384]

kernels	dynamically generated based on voxel spacings [[3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3]]
strides	dynamically generated based on voxel spacings [[1, 1, 1], [2, 2, 2], [2, 2, 2], [2, 2, 2], [2, 2, 2], [2, 2, 2]]

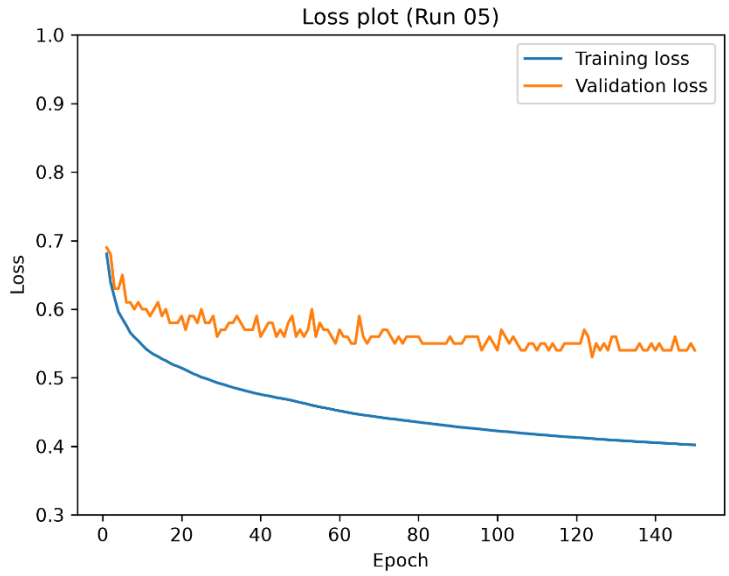
Experiment results

Metrics

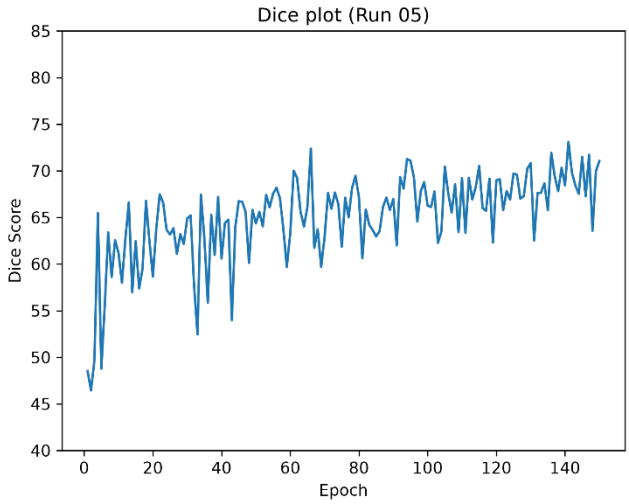
Dice	**Val loss**	**Train loss**	**D1**	**D2**	**D3**	**Best epoch**
73.10	0.269	0.402	73.95	64.49	80.86	140/150

- Note that this isn't the best dice score for region 3, i.e. ET. The highest score we get is 83.82. But again, we see much variation within the subregions since D2 is quite bad, and D1 is also not as good
- Dice score is slightly better, i.e. similar, to dice score for when filter sizes were larger. Also notice the best epoch is higher, indicating that the model is not overfitting so much

Loss plot:

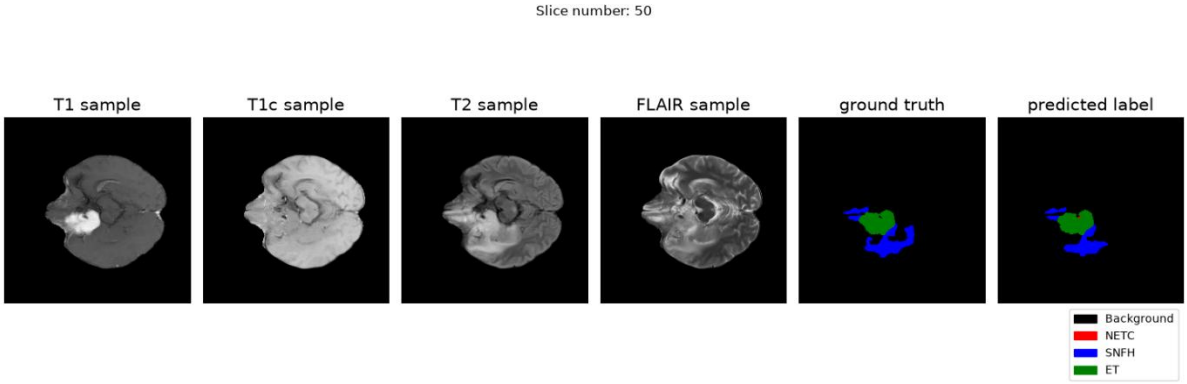


Dice plot:



Predictions analysis

Like the DynUNet model, the BraTS model sometimes struggles to identify some specific shape



Run 06

Run objectives and summary

- The purpose of this run was to test the DynUNet again, this time with larger filter sizes at the start and the use of res blocks. This was to try and improve its ability to identify local details
- The overall dice score did not improve. However, the model got better at identifying the ET (green) region. Its ability to identify the SNFH (blue) and NETC (red) regions got worse though. I am not sure if these were due to the addition of res blocks or if it was due to a change in the filter size

Model Configuration

Backbone	**BraTS 22 Model**
input shape	(5, 145, 162, 133)
output channels	3 (for each of the tumour subregions)
filters	[96, 96, 128, 192, 256, 320] (Compared with Run02: [**64**, 96, 128, 192, 256, **384**])

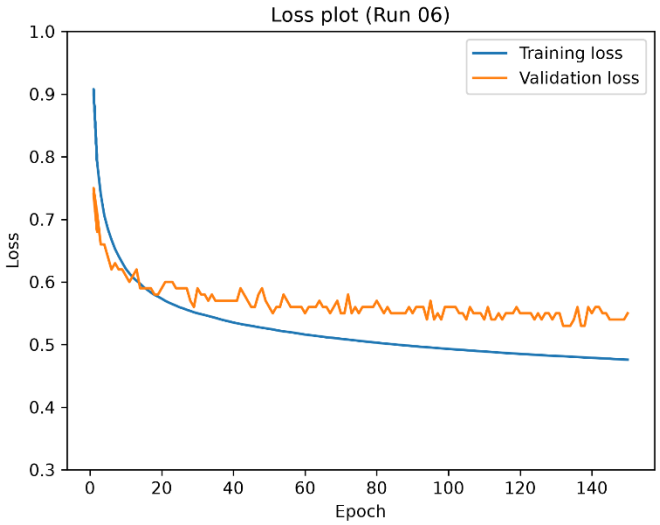
kernels	dynamically generated based on voxel spacings [[3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3]]
strides	dynamically generated based on voxel spacings [[1, 1, 1], [2, 2, 2], [2, 2, 2], [2, 2, 2], [2, 2, 2], [2, 2, 2]]
res_block	True

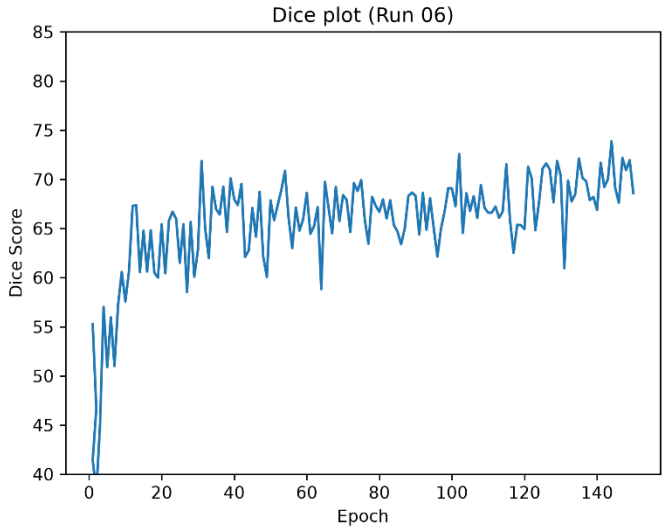
Experiment results

Metrics

	Dice	**Val loss**	**Train loss**	**D1**	**D2**	**D3**	**Best epoch**
Run 06	73.88	0.2612	0.4759	72.68	68.63	80.32	143/150
Run 02	76.02	0.56	0.4137	76.53	74.1	77.43	97/100

Loss and Dice plot





Run 07

Run objectives and summary

- Since adding res blocks and increasing top layer filter sizes didn't cause the model to improve, we can check what only increasing the filter sizes will do (not using res blocks)

Model Configuration

Backbone	**BraTS 22 Model**
input shape	(5, 145, 162, 133)
output channels	3 (for each of the tumour subregions)
filters	[96, 128, 128, 192, 256, 384] (Compared with Run06: [96, 96, 128, 192, 256, 320]) (Compared with Run02: [64, 96, 128, 192, 256, 384])
kernels	dynamically generated based on voxel spacings [[3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3]]

strides	dynamically generated based on voxel spacings [[1, 1, 1], [2, 2, 2], [2, 2, 2], [2, 2, 2], [2, 2, 2], [2, 2, 2]]
res_block	False

Training Configuration

epoch number	170
batch size	2

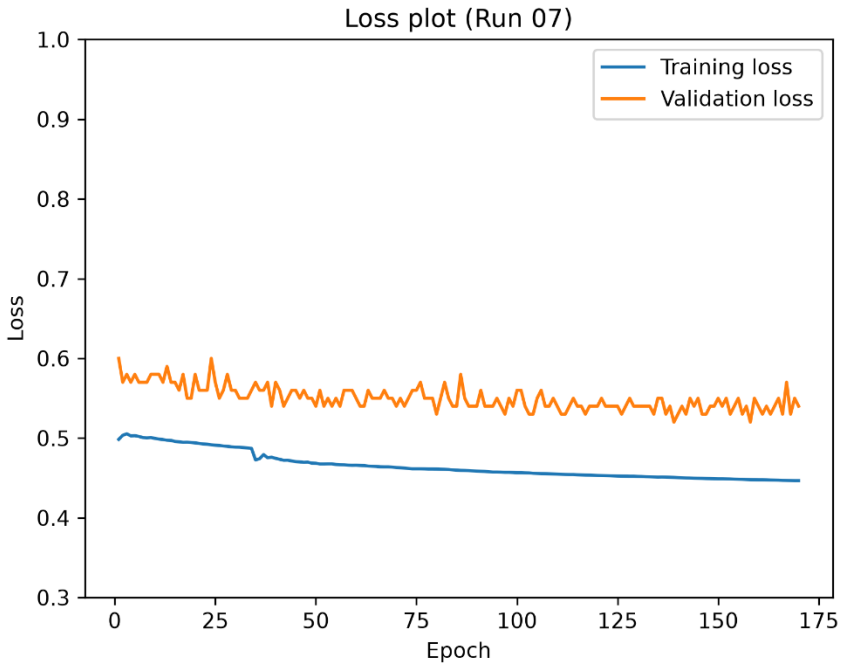
Experiment results

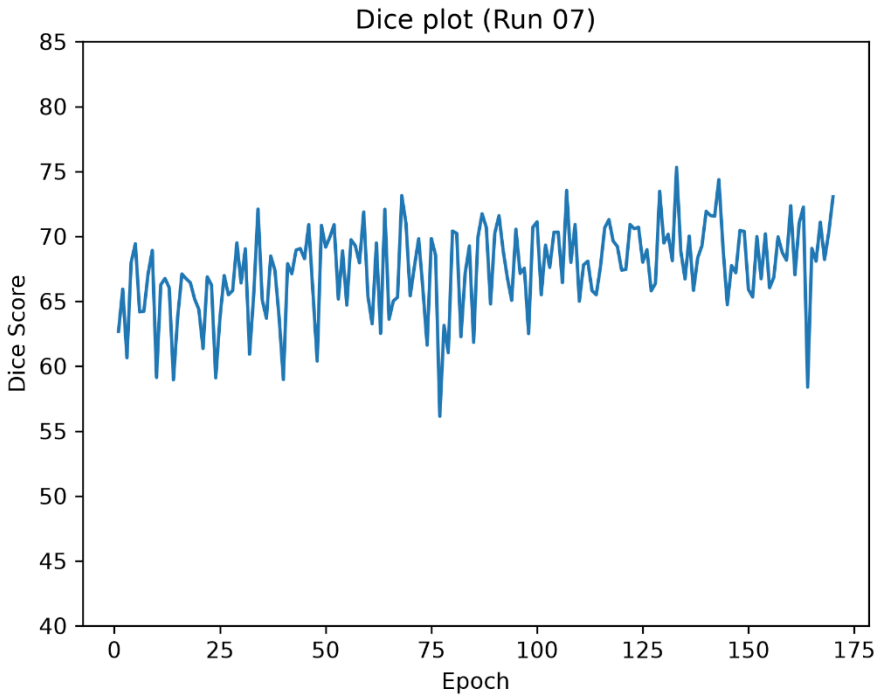
Metrics

	Dice	**Val loss**	**Train loss**	**D1**	**D2**	**D3**	**Best epoch**
Run 07	75.32	0.2468	0.4467	73.40	70.42	82.14	132/170
Run 02	76.02	0.56	0.4137	76.53	74.1	77.43	97/100

From looking at the metrics, the model has improved in being able to identify the D3 region (i.e. ET, shown in blue) accurately. However, this seems to be at the cost of D1 and D2 being lower

Loss and Dice plots





Run 08

Run objectives and summary

- Kept same configurations as Run 02, except added Res blocks (and used epoch 150)
- This did not cause any significant decrease or increase in performance

- However, the gap between val loss and train loss was larger in the end

Model Configuration

Backbone	**BraTS 22 Model**
input shape	(5, 145, 162, 133)
output channels	3 (for each of the tumour subregions)
filters	[64, 96, 128, 192, 256, 384] (Compared with Run02: [64, 96, 128, 192, 256, 384])
kernels	dynamically generated based on voxel spacings [[3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3]]
strides	dynamically generated based on voxel spacings [[1, 1, 1], [2, 2, 2], [2, 2, 2], [2, 2, 2], [2, 2, 2], [2, 2, 2]]
res_block	True (Compared with Run02: False)

Experiment results

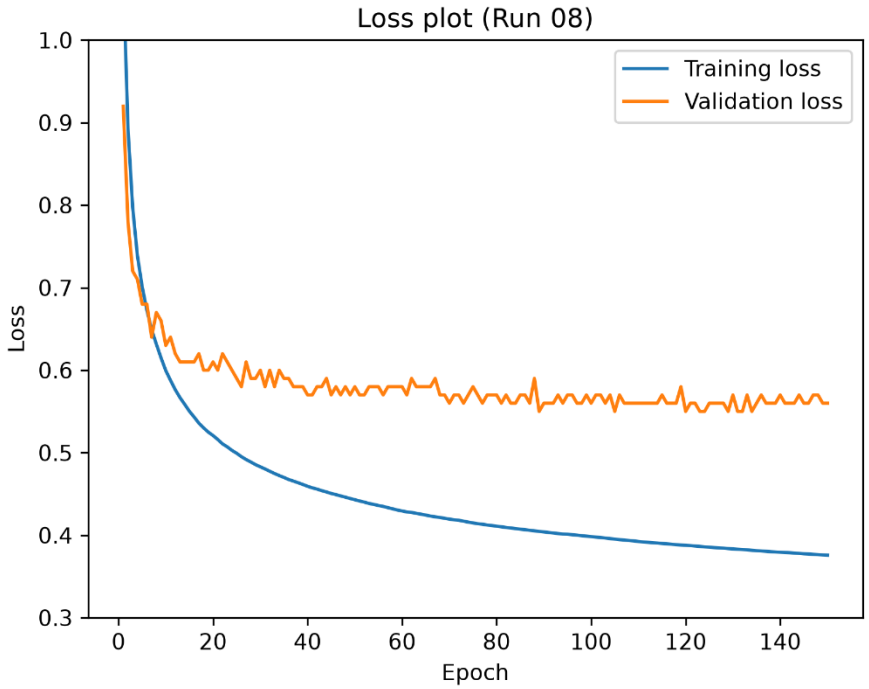
Metrics

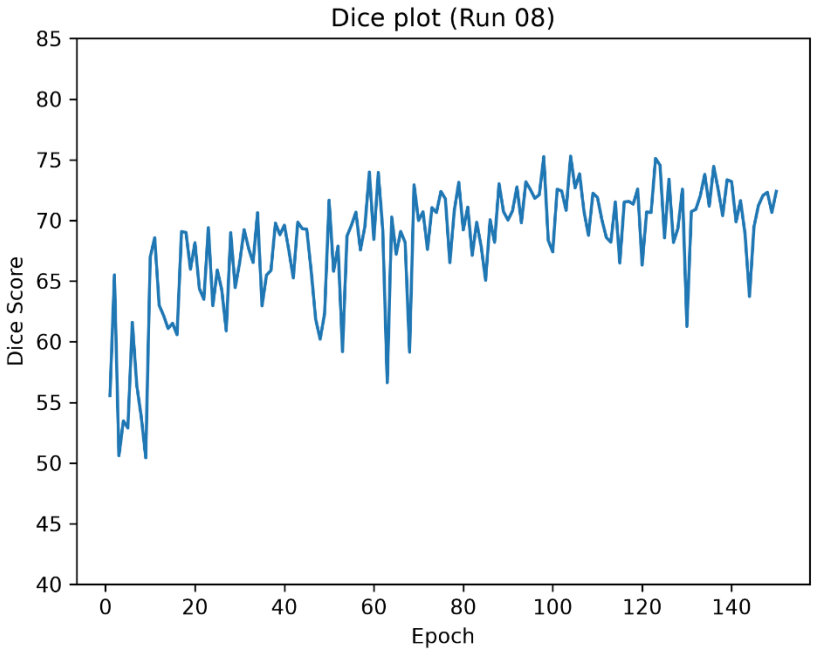
	Dice	**Val loss**	**Train loss**	**D1**	**D2**	**D3**	**Best epoch**
Run 08	75.30	0.247	0.3758	70.52	77.56	77.83	132/150

Run 02	76.02	0.56	0.4137	76.53	74.1	77.43	97/100

- Notice the better performance on the D2 region, i.e. SNFH, represented in Blue
- Performance on D3 region, i.e. ET, represented in Green, is similar
- Drop in performance for D1 region, i.e. NETC, represented in Red (which took up a much smaller percentage of the data)

Loss and Dice plots




Dice plot (Run 08)
Dice Score
Epoch
85
80
75
70
65
60
55
50
45
40
0
20
40
60
80
100
120
140

Appendix C. A little bit of fun

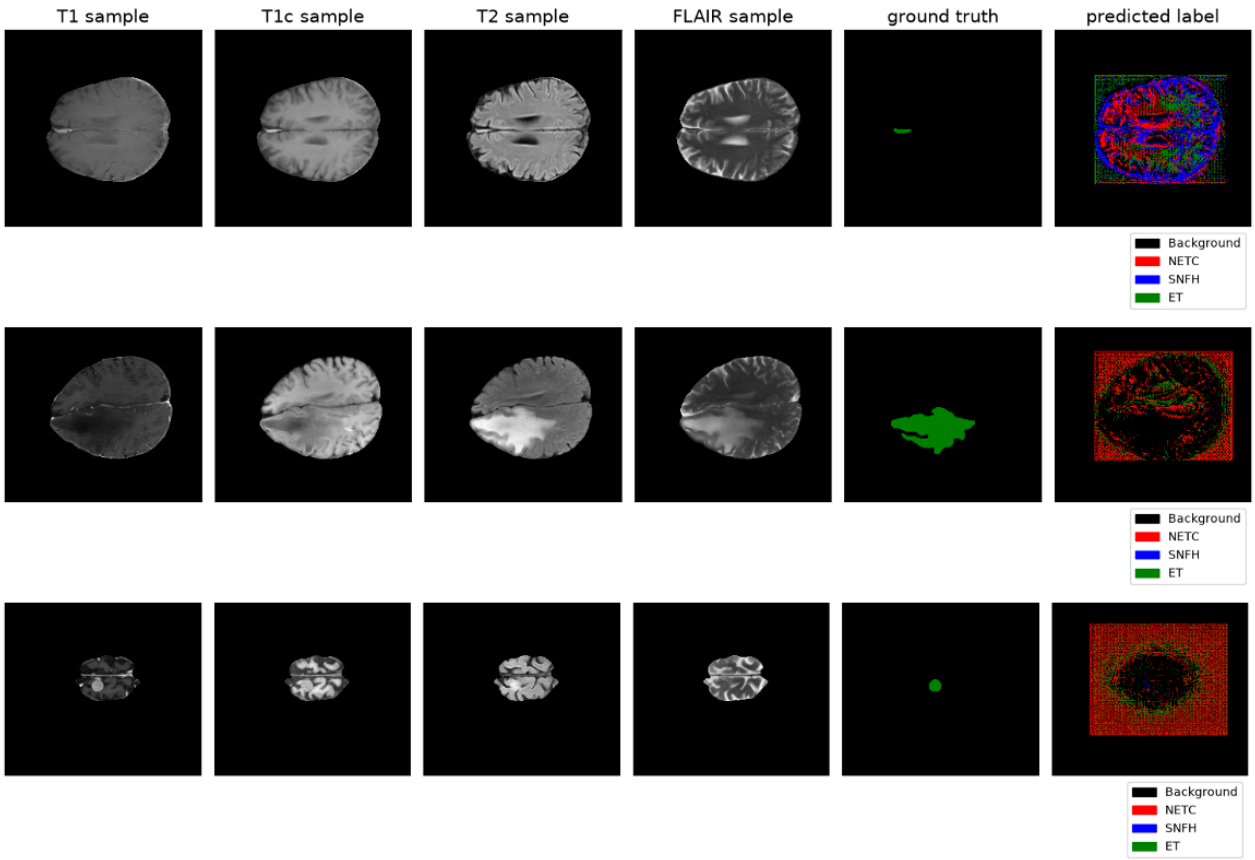


Figure 19. A series of slightly scary plots that I got when I accidentally ran inference on a model with the wrong configuration.

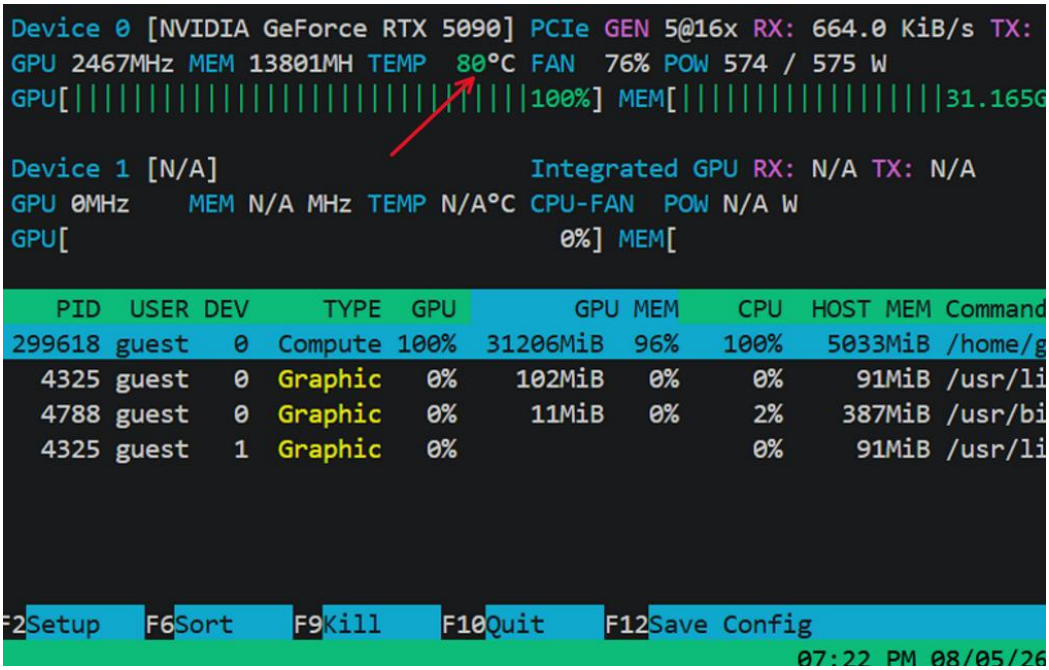


Figure 20. When training, I noticed that the GPU was at 80 degrees! Thankfully the maximum safe temperature is 90 degrees.

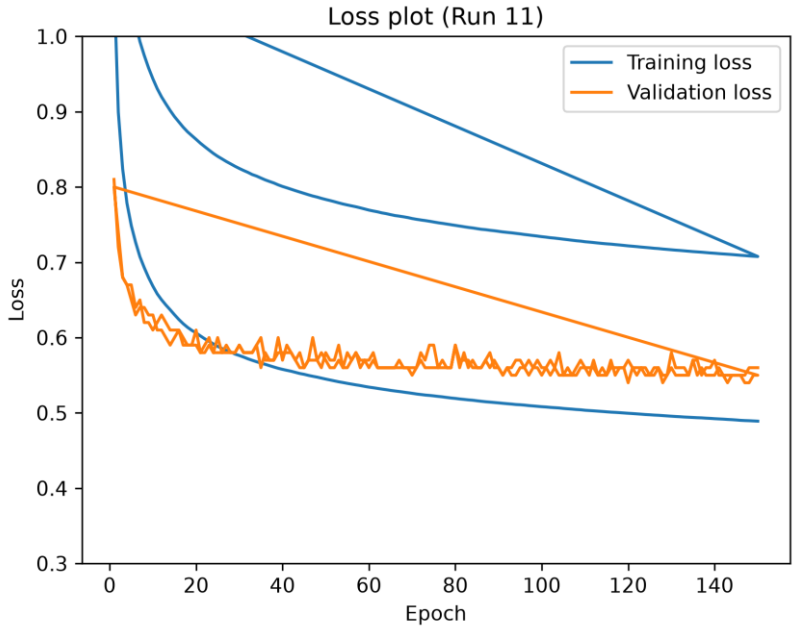


Figure 21. A strange loss plot I got when I accidentally plotted the loss from two models at the same time.